

衛生薬学 学習プロジェクト 運用マニュアル（教員・管理者用）

順天堂大学 薬学部 衛生化学分野 最終更新: 2026-06-24

このマニュアルについて このプロジェクト一式（学生用アプリ・管理画面・問題データ・弾幕機能）を、**あなた自身が運用・管理するための**手引きです。中身はAIアシスタント（Claude）と一緒に作り上げてきたため、「どこに何があり、何がどう動いているか」をまず把握できるようにまとめました。プログラミングの知識は前提にしていません。困ったら、このマニュアルの該当箇所をClaudeに見せて「ここをやって」と頼めば大丈夫です。

目次

- まず全体像（このプロジェクトの3つの顔）
- 公開URLとファイルの早見表
- 学生用アプリ（index.html）
- 管理画面（admin.html）で学生の声に対応する
- 問題を直す・追加する
- データとファイルの地図
- 弾幕機能（授業中のコメント表示）
- 公開・更新のしくみ（変更が学生に届くまで）
- 困ったとき / よくある質問
- 用語集（やさしい言葉で）

1. まず全体像（このプロジェクトの3つの顔）

このプロジェクトは、見た目は別々ですが、**3つの道具**が組み合わさってできています。

#	道具	誰が使う	何をする
①	学習アプリ （index.html）	学生	過去問・予想問題を解いて学ぶ
②	管理画面 （admin.html）	先生（あなた）	学生の報告を確認し、問題を「確認済み」にする
③	弾幕 （danmaku_*.html）	授業中の学生＋先生	授業スライドにコメントを流す

データの置き場所は2か所だけ

むずかしく見えますが、データのありかは実は2種類です。

【1】GitHub（ギットハブ）… ファイルの保管庫 & 公開サーバー

- ・問題データ（jsonlファイル2つ）
- ・画像（FUTURE/ と JPPE_norm/）
- ・アプリ本体（index.html, admin.html, 弾幕ページ）
→ これらは「GitHub Pages」という機能で、そのままWebサイトとして公開されている

【2】Supabase（スーパベース）… クラウドのデータベース（2プロジェクト）

- ・本体用：学生の「報告」と「検証済み」の記録、管理画面のログイン
- ・弾幕用：授業中のコメント（※こちらは別プロジェクト）

ポイント： - **問題そのもの**（問題文・選択肢・正解・解説・画像）は **GitHub 中のファイル**にあります。
Supabaseには入っていません。 - **学生の声**（報告・「妥当」評価）と**検証の記録**だけが **Supabase** に貯まります。
- だから「問題を直す」=GitHubのファイルを直す、「学生の報告を見る」=Supabaseを見る（=管理画面を開く）、という住み分けになっています。

2. 公開URLとファイルの早見表

公開されているWebページ（GitHub Pages）

ベースURL： https://tmatsuka-art.github.io/eisei_yakugaku/

ページ	URL	用途
学習アプリ	.../eisei_yakugaku/index.html （または末尾なし）	学生に配るメインのURL
管理画面	.../eisei_yakugaku/admin.html	先生用（ログイン必須）
弾幕・投稿	.../eisei_yakugaku/danmaku_post.html	授業中、学生がコメントを送る
弾幕・表示	.../eisei_yakugaku/danmaku_screen.html	授業中、コメントを流す投影画面

よく触る大事なファイル（GitHubリポジトリ内）

ファイル	中身
<code>future_questions.jsonl</code>	予想問題 約1,408問（F001～）の本体
<code>hygiene_with_images_v4.jsonl</code>	過去問 648問（97～111回）の本体
<code>lecture_map.json</code>	どの問題をどの授業回で出すかの割り当て表
<code>index.html</code> / <code>admin.html</code>	学習アプリ／管理画面の本体
<code>CLAUDE.md</code>	Claude（AI）向けの作業ルール集。 Claudeは作業前に必ずこれを読む
<code>FUTURE/</code> フォルダ	予想問題の図（PNG画像 約70枚）
<code>JPPE_norm/</code> フォルダ	過去問の図（回ごとのフォルダ JPPE097～111）
<code>by_subcat_md/</code> フォルダ	分野別に問題を読みやすく書き出したMarkdown（49ファイル）

ファイル名に `.bak` が含まれるものは「作業前の控え（バックアップ）」です。普段は触りません（→ 6章）。

3. 学生用アプリ（index.html）

学生が実際に解く画面です。学生向けの詳しい使い方は別冊「**衛生薬学アプリ 使い方ガイド（PDF）**」にまとめてあります。ここでは管理者として知っておく要点だけ。

- **できること**：分野・国試の回・授業回・予想/過去問で範囲を選んで出題、即採点と解説、苦手問題のみ復習、検索、進捗確認、問題への「報告」。
- **ログイン不要**：学生は匿名で使えます。学習の記録（正答率など）は**その学生の端末のブラウザの中だけ**に保存され、サーバーには送られません。
- **データの読み込み元**：開くと `future_questions.jsonl` と `hygiene_with_images_v4.jsonl` を読み込み、画像は `FUTURE/` や `JPPE_norm/` から表示します。**つまり、これらのファイルをGitHubで更新すれば、学生のアプリにも反映されます**（→ 8章）。
- **学生が送れるもの**：各問題の「👍 OK（妥当）」「△ 気になる点を報告」。これだけがSupabaseに送られ、あなたの管理画面に届きます。

4. 管理画面（admin.html）で学生の声に対応する

学生からの報告を確認し、対応し、問題を「確認済み」にしていく、**先生専用の画面**です。

4-1. 開き方とログイン

1. ブラウザで `https://tmatsuka-art.github.io/eisei_yakugaku/admin.html` を開きます。 - △ **必ずこの公開URLから開いてください**。パソコンに保存した `admin.html` を単独でダブルクリックして開くと、問題データ

(jsonl)を読み込めず、問題の中身が「見つかりません」になることがあります。

2. 「ログイン」画面が出ます。**メールアドレスとパスワード**を入れて「ログイン」。- これはSupabaseに登録した管理者アカウントです（あなたが設定したもの）。- 失敗すると赤字で「ログイン失敗」と出ます。
3. 一度ログインすれば、しばらくはログインしたままになります。やめるときは右上の「ログアウト」。

4-2. 5つのタブ

画面の上に5つのタブがあり、右の **(数字)** は件数です。

タブ	表示されるもの	できること
△ 未対応の報告	まだ対応していない報告	内容確認 → 対応済みにする／修正依頼コピー／検証済みにする
✓ 対応済み	対応を終えた報告	内容確認／未対応に戻す
全報告	すべての報告	上記の操作
📋 検証履歴	確認済みの問題一覧	検証の解除／学生人気の問題を「公式検証に格上げ」
🔧 問題を直す	問題の検索ボックス	報告が無くても、任意の問題を探して修正依頼を作る

4-3. 報告に対応する（基本の流れ）

「△ 未対応の報告」タブで、問題ごとに報告カードが並びます。カードには**問題の中身**と、各報告の**種別**（「正解が違うと思う」「選択肢が不適切」「問題文がわかりにくい」「図表がおかしい」「解説が不十分」「その他」）、**日時**、**学生の匿名ID**、**コメント**が出ます。

対応の手順：1. 報告を読む。2. 直す必要があれば、後述の「📋 Claudeに修正依頼をコピー」で直す（→ 5章）。3. 「**対応の結果**」をプルダウンから選ぶ：- **fixed**（修正した）／**not_an_error**（誤りではなかった）／**wontfix**（今回は直さない）／**duplicate**（重複）4. 必要ならメモを書いて、「✓ **対応済みにする**」を押す。5. これで「対応済み」タブに移ります。間違えたら「△ 未対応に戻す」で戻せます。

知っておくと安心な自動動作：対応の結果に **fixed** を選ぶと、その問題は自動で「**検証済み（ヒトの目で確認済み）**」にもなります。「直した＝確認した」とみなす仕組みです。ただし、その後「未対応に戻す」をしても、**いったん付いた検証済みは自動では外れません**。外したいときは下記の検証ボタンで解除してください。

4-4. 「検証済み」を管理する

「検証済み」は、**その問題を人の目で確認した／妥当だ、という印**です。問題カードや検証履歴に、次の3つのバッジで状態が出ます。

バッジ	意味
✓ ヒトの目で確認済み （緑）	先生が確認した。いちばん信頼できる状態
🤖 AI作成・検証募集中 （青）	まだ誰も確認していない
👤 学生が「妥当」と評価のみ （オレンジ）	先生の確認はまだだが、学生が「妥当」と投票している

操作： - 問題カードの「 検証済みにする」で確認済みにできます（即実行）。「 検証済み（解除）」で外せます（確認あり）。 - 「 検証履歴」タブで、学生人気だけの問題（オレンジ）を「 公式検証に格上げ」で先生のお墨付き（緑）に引き上げられます。

4-5. 「 Claudeに修正依頼をコピー」

このボタンは**問題の中身を自動でコピー**してくれるので、手で書き写す必要がありません。コピーされた文章を**Claude（このAIアシスタント）に貼り付けて「直して」と頼む**のが基本の流れです。

- **報告カードの中**のボタン → 学生の報告内容＋問題の現在の中身＋「直して」という依頼文がコピーされます。
- 「 **問題を直す**」タブのボタン → そのカードの「**どう直したいか**」の入力欄に書いた指示も一緒にコピーされます。さらに、過去問の改題ルール（IDは変えない・タイトルに「(改)」を付ける等）も自動で付きます。

このボタンは**依頼文を作るだけ**で、問題ファイルは書き換えません。実際の修正はClaudeが行います。

4-6. 毎朝の自動まとめ（GitHub Issue ダイジェスト）

毎朝**8時（日本時間）**に、過去24時間に届いた学生の報告を**自動でまとめ、GitHubの「Issue（イシュー）」として1件作成**する仕組みが動いています（メールではありません）。

- GitHubリポジトリ [tmatsuka-art/eisei_yakugaku](https://github.com/tmatsuka-art/eisei_yakugaku) の「Issues」タブを見ると、 **学生報告ダイジェスト 2026/MM/DD（N件）** のような項目が並びます。
- 中身は、報告のあった問題・件数・種別・コメントの一覧と、管理画面へのリンク。
- これを見れば、毎朝「昨日どんな報告が来たか」をまとめて把握できます。報告が0件の日はIssueは作られません。

5. 問題を直す・追加する

問題の中身を直すのは、**あなたがClaude（AI）に頼み、Claudeがファイルを書き換える**という二人三脚で行います。あなたが直接ファイルを編集する必要はありません。

5-1. 直すきっかけは2通り

(a) 学生の報告から直す 1. 管理画面で報告を開く。 2. 「 Claudeに修正依頼をコピー」を押す。 3. Claudeに貼り付けて「直して」と頼む。 4. Claudeが該当ファイルを修正し、保存（コミット＆プッシュ）。 5. 管理画面でその報告を「対応済み（fixed）」にする。

(b) 自分で気づいて直す - Claudeに直接「**F856の選択肢3を直して**」のように頼むだけでもOK。問題番号と直したい点が分かれば十分です。 - きちんと依頼したいときは「 **問題を直す**」タブで対象を検索し、指示を書いてコピー → Claudeへ。

5-2. まとめて点検する「AIレビュー」（モードA / B / C）

個別の修正とは別に、問題を10～20問単位でまとめて点検・改善する作業があります。複数の専門の係（コアカリ係・過去問係・事実確認係・品質係など最大7係）が並列で点検し、見つかった改善案を後述のTierに仕分けします。

頼み方は3モード。**ふだんは何も言わずモードA**です。

モード	あなたの言い方	違い
● モードA （標準）	「 次のバッチやって 」	安全な変更は自動、重要な変更は確認を求められる（判断は1バッチ3〜5回程度）
● モードB （高速）	「 モードBで頼む 」	表現の磨き込みも自動。確認は重要案件だけ
● モードC （背景）	「 モードCで頼む 」	全部処理して、重要案件だけ後でまとめて判断

5-3. 何を確認させられる？（Tier 1・2・3）

改善案は重要度で3段階に仕分けられます。**あなたが判断を求められるのは Tier 2 と Tier 3 だけ**です。

段階	内容	あなたの関与
Tier 1	解説の追記、分類コードの軽微な修正、誤字直しなど	なし （自動反映・事後報告のみ）
Tier 2	攪乱肢の表現改善、絶対化表現の除去、正答位置の調整など	1回 （「全部OK」か「この番号は却下」）
Tier 3	問題文の全面書き換え、削除、正解そのものの変更など	1問ずつ 確認

迷ったら上の段階に上げる（＝確認を求める）ルールにしています。「勝手に直された」を防ぐためです。

5-4. 安全のしくみ（失敗してもやり直せる）

- **バックアップを必ず取る**：jsonlを直す前に、Claudeは `future_questions.jsonl.bak_pre_oo_日付` のような控えを作ります。名前を見れば「いつ・何の作業の前か」が分かります。
- **直接いきなり編集しない**：必ず控えを取ってから。これはCLAUDE.mdに明記された鉄則です。
- **1つの修正＝1つの記録**：変更はGitHubの履歴（コミット）に1件ずつ残るので、後から「いつ何を直したか」をたどれ、必要なら戻せます。
- **承認した分だけ反映**：AIレビューでは、承認された内容をいったん決定ファイル（decisions）に貯め、専用スクリプトがまとめて反映します。Claudeが思いつきで書き換えることはありません。

5-5. 問題をまとめて追加・分類する

過去には、問題をまとめて追加したり、授業回（`lecture_map.json`）へ割り当て直したりする作業も行ってきました（`_work/` フォルダのスクリプト群がその痕跡）。

△ 大事な前提：問題に元から付いている分類ラベルは**3〜4割が誤り**であることが分かっています。そのためラベルを鵜呑みにせず、**問題の中身を読んで1問ずつ判定**するのが、このプロジェクトで確立したやり方です。新しく分類・追加するときは、Claudeにもこの方針で進めてもらってください。

6. データとファイルの地図

6-1. 「jsonl」って何？

`.jsonl` は **1行に1問**が入ったテキストファイルです。各行が「項目名: 中身」の集まり（JSON形式）になっていて、Excelの表を縦に積んだようなイメージです。日本語が含まれるので、メモ帳で開くと文字化けすることがあります。直接いじらず、**Claudeに頼んで編集する**のが安全です。

6-2. 2つの問題ファイル

① `future_questions.jsonl`（予想問題 約1,408問・F001～）

1問に含まれる主な項目：

項目	中身
<code>problem_id</code>	問題の背番号（予想問題は必ず <code>F</code> で始まる）
<code>problem_text</code>	問題文
<code>choices</code>	選択肢（図問題は <code>['IMAGES']</code> という目印）
<code>answer</code>	正答の番号（複数選択もあり）
<code>comment</code>	解説
<code>new_corecurri_code</code>	新コアカリ（令和4年改訂）の分類コード
<code>corecurri_code</code>	旧コアカリのコード
<code>assets</code>	図ファイルへのパス（図なしは空）
<code>lecture</code> 関連 / <code>lecture_map.json</code>	どの授業回で出すか
<code>ai_review</code>	AIレビューの履歴。 この項目が無い問題＝まだ点検していない目印

② `hygiene_with_images_v4.jsonl`（過去問 648問・97～111回）

項目はほぼ共通ですが、過去問ならではの： - `problem_id` は **回 + 問番号の6桁**（例 `097016` = 第97回・問16） - `year` は国試の回数（97～111。予想問題は `0`） - `display_title` は「第97回 衛生 問16」のような表示名 - 図のある問題は189問。 `assets` が `JPPE_norm/JPPE097/097123.png` のように画像を指す

6-3. `lecture_map.json`（授業回の割り当て表）

「どの問題を、どの科目の第何回で出すか」を決める表です。学習アプリの「授業回別」表示の土台になります。

- 3科目（**衛I＝衛生化学I／衛II＝衛生化学II／公衆＝公衆衛生学**）に分かれる
- 各科目の中に授業回（1, 2, 3…）があり、回ごとに**テーマ**と**問題IDのリスト**が入る
- 問題を授業から外したいときは、この表からIDを抜きます（jsonlの問題自体は残る）

6-4. 画像の置き場所

種類	場所
予想問題の図	<code>FUTURE/</code> フォルダ（ファイル名 = problem_id、例 <code>F002.png</code> ）
過去問の図	<code>JPPE_norm/JPPE097</code> ～ <code>JPPE111</code> （回ごとのフォルダ）

画像は**すべてGitHubの中**にあります。Supabaseには置いていません。

6-5. `by_subcat_md/`（分野別の読みやすい問題集）

jsonl（機械向け）の中身を、**分野ごとに人が読みやすく書き出した**Markdown（49ファイル）です。点検や印刷に使えます。これは元データ（jsonl）から作り直せる「派生物」なので、ここを直しても本体には反映されません。**直すのは必ずjsonl側**です。

6-6. Supabase（クラウドのデータベース）

本体のSupabaseプロジェクト（`gmkrsgqgndlrabiicvkh.supabase.co`）には、**2つの表**だけがあります。

表	中身
<code>question_feedback</code>	学生の「👍 妥当」評価と「△ 報告」。報告には種別・コメント・日時・匿名IDが入る。対応すると先生のメモや結果も追記される
<code>question_verifications</code>	「検証済み」の記録（問題ID・検証日時・手動か報告経由か・メモ）

そのほか、ログイン認証（管理画面用）と、毎朝のダイジェスト自動作成（4-6）もここで動いています。

6-7. 控え・記録ファイルの見分け方

種類	例	何か
バックアップ	<code>*.bak_pre_oo_日付</code>	作業前の控え。普段触らない。新しい日付ほど最近
レビュー進捗	<code>ai_review_log_第9回.md</code> 等	どの問題を点検したかのチェック表
引き継ぎメモ	<code>HANDOFF_2026-06-10.md</code> 等	前回作業の引き継ぎ。 新しい日付のものが最新
経緯まとめ	<code>検証経過まとめ.md</code> 、 <code>新コアカリ分類案.md</code>	これまでの方針・分類ルール

新しいセッションでClaudeに作業を頼むときは、「最新のHANDOFFを読んでから始めて」と言うと、前回の続きをスムーズに引き継ぎます。

7. 弹幕機能（授業中のコメント表示）

授業スライドの上に、学生がスマホから送ったコメントを、ニコニコ動画のように右から左へ流す機能です。匿名なので学生が気軽に反応・質問でき、大教室でも双方向のやりとりが生まれます。

△ 弹幕のデータは、学習アプリとは別のSupabaseプロジェクト（[eisei-quiz-live](#)）の `danmaku_comments` という表に貯まります。学習アプリの報告とは混ざりません。

7-1. 3つのページの役割

ページ	役割	開き方
<code>danmaku_post.html</code>	学生が投稿する画面	スマホで <code>.../danmaku_post.html</code> （QRから自動で開ける）
<code>danmaku_screen.html</code>	コメントを流す投影画面	OBS用は <code>.../danmaku_screen.html?transparent=1</code>
<code>弹幕デモ.html</code>	練習用（ネット不要、その場で試せる）	ファイルを直接開く

部屋（room）：授業ごとにコメントを分けたいときは、URLに `?room=授業名` を付けます（投稿側・表示側で同じ名前にする）。投影画面のQRコードは自動で同じ部屋に紐づくので、学生がQRを読めば自動的に同じ部屋に入ります。

7-2. 授業当日の流れ

OBS（無料の画面合成ソフト）で「PowerPointの画面」と「透明な弹幕ページ」を2枚重ねて投影します。

1. **PowerPoint**を「ウィンドウ表示のスライドショー」で開く（スライドショーの設定 → 「出席者として閲覧する（ウィンドウ表示）」）。送りは → キー / Page Down / ホイール。
2. **OBSを起動**し、弹幕が上・PowerPointが下の重ね順、右下にQRが出ているか確認。
3. **投影**：教室なら「全画面プロジェクター」、Zoomなら「仮想カメラ開始」→ ZoomでOBS Virtual Cameraを選ぶ。
4. **学生に投稿してもらう**：QRを読んでもらう。投稿は3秒に1回まで。
5. **終了**：OBSの全画面/仮想カメラを止め、PowerPointを Esc で終了。

弹幕が出ない＝故障とは限りません。 弹幕は「開いた後に届いた新しいコメント」を流します。起動直後に直近のコメントを少し流したあとは、新しい投稿が来るまで画面はQRだけになります（正常）。出ないときは、まず投稿側と表示側の**部屋(room)**が一致しているかを確認してください。

7-3. OBS設定の要点

詳しくは [弹幕_OBS設定ガイド.md](#) と [弹幕_授業当日の操作手順.md](#) にあります。要点は： - OBSで**ブラウザソース**を追加し、URLに `.../danmaku_screen.html?transparent=1` を指定（幅1920・高さ1080）。 - `?transparent=1` を付けると弹幕ページの背景が透明になり、下のPowerPointが透けます（クロマキー不要）。 - もし弹幕の背景が黒くてスライドが隠れるときは、**URL末尾の `?transparent=1` が付いているか**を確認。

補足：当日手順メモには「通信: SUBSCRIBED」表示や「🔄再読み込み」ボタンの記述がありますが、現在のページには無い可能性があります。実機のOBSで一度表示を確認しておくとう安心です。

8. 公開・更新のしくみ（変更が学生に届くまで）

このサイトは **GitHub Pages** で公開されています。仕組みはシンプルです。

ファイル直す (Claudeが編集)
↓
GitHubに保存 (コミット & プッシュ = Claudeが実行)
↓
数十秒～数分で、公開URLに自動反映
↓
学生がページを再読み込みすると最新の問題が見える

- 問題を直したり追加したりすると、**GitHubに保存された時点で自動的に公開サイトにも反映**されます。特別な「アップロード」操作は要りません。
- 学生側は、**ページを再読み込み（リロード）**すれば最新になります。
- 「ちゃんと反映されたかな？」と思ったら、公開URL（`.../index.html`）を開いて確認できます。

このプロジェクトは **main（メイン）** という**1本の流れに直接保存**する運用です。Claudeに「直して」と頼むと、修正 → 保存 → 公開反映まで一通りやってくれます。

9. 困ったとき / よくある質問

困りごと	どうする
管理画面で問題の中身が「見つかりません」	保存したファイルを直接開いていませんか。必ず公開URL <code>.../admin.html</code> から開いてください
管理画面にログインできない	メール／パスワードを確認。分からなくなったらSupabaseの管理者設定の問題なので、Claudeに相談を
学生が「答えが違う」と報告してきた	管理画面で「📄 Claudeに修正依頼をコピー」→ Claudeに貼って確認・修正を依頼
問題を直したのに学生の画面が古い	学生にページの再読み込みを案内。数分待っても反映されなければClaudeに確認を依頼
直したら別のところが壊れた気がする	慌てず、控え（ <code>.bak_pre_...</code> ）があります。「最新のバックアップから戻して」とClaudeに頼めます
弾幕が流れない	投稿側・表示側の <code>room</code> が一致しているか確認。起動直後に出ないのは正常
何から手をつければいいのか分からない	このマニュアルの該当章をClaudeに見せて「ここをやりたい」と伝えれば、一緒に進められます

10. 用語集（やさしい言葉で）

言葉	意味
GitHub（ギットハブ）	ファイルの保管庫。変更の履歴が全部残り、いつでも戻せる。ここに置いたファイルはそのままWebで公開もできる
GitHub Pages	GitHubの中身をそのままWebサイトとして公開する機能。このプロジェクトの公開URLはこれ
コミット / プッシュ	「変更を保存する」「保存をGitHubに送る」操作。Claudeが代わりに行う
リポジトリ	このプロジェクト一式が入っているGitHub上の入れ物（ <code>tmatsuka-art/eisei_yakugaku</code> ）
Supabase（スーパベース）	クラウドのデータベース。学生の報告や検証記録、弹幕コメントが貯まる
jsonl	1行に1問が入ったテキストのデータ形式。問題の本体
コアカリ / new_corecurri_code	薬学教育のモデル・コア・カリキュラムの分類。問題がどの項目に当たるかのコード
検証済み	人の目で確認した、という印。Supabaseに記録される
攪乱肢（かくらんし）	誤答の選択肢。良問にするための作り方ルールがCLAUDE.mdにある
Issue（イシュー）	GitHubの「やること・気づきメモ」機能。毎朝の報告まとめがここに自動で立つ
OBS	画面を重ねて配信・録画する無料ソフト。弹幕の投影に使う
room（ルーム）	弹幕の「部屋」。授業ごとにコメントを分ける合言葉
CLAUDE.md	Claude（AI）への作業指示書。Claudeは作業前に必ず読む。運用ルールを変えたいときはここを更新する

このマニュアルの更新について 運用を変えたとき（新しい機能を足した、手順を変えた等）は、このファイル（`運用マニュアル_教員用.md`）をClaudeに更新してもらってください。「マニュアルの○章を最新に直して」と頼めば、内容を反映してPDFも作り直せます。